

Actividad evaluable UT2A4

CRUD con acceso a base de datos

Enlace a Github:	1
Procedimiento para la actividad	1
Creación de una nueva rama	1
Conexión del login con la base de datos	2
Creación del backend	4
Programar los endpoints del backend	5
Llamar endpoints desde el frontend	6
Comprobación del código	7
Creación de menú para todas las páginas	8
Creación del formulario en Dashboard.tsx	9
Lista para los datos y su eliminación	11
Creación del fichero backend/services/items.js	12
Añadir funcionalidad	14

Enlace a Github:

<https://github.com/Hector5472/ProyectoHFM/tree/crud>

Procedimiento para la actividad

Creación de una nueva rama

Primero, creo una nueva rama llamada crud, situandome en la carpeta "ProyectoHFM", escribiendo en la consola el siguiente comando:

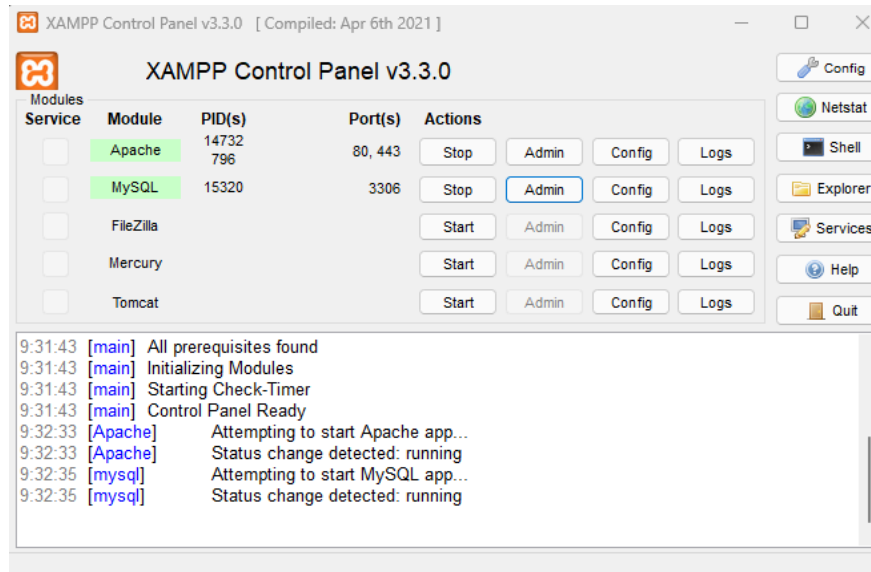
```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM> git checkout -b crud  
Switched to a new branch 'crud'
```

Y compruebo que estoy situado en la rama creada.

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM> git branch  
* crud  
login  
main  
redux
```

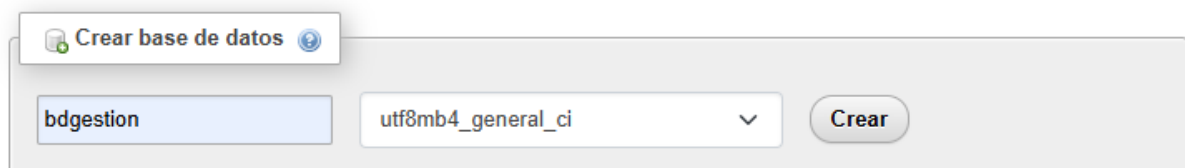
Conexión del login con la base de datos

Con XAMPP, inicio el servicio de Apache y MySQL.

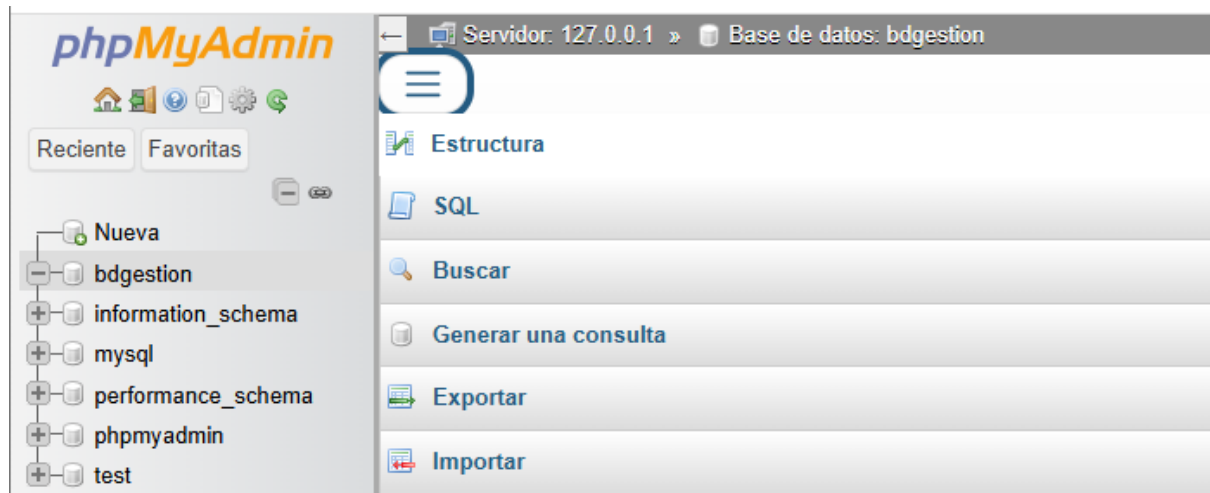


Me dirijo al administrador de base de datos picando en el botón Admin del servicio MySQL y allí creo una nueva base de datos con el nombre bdgestion.

Bases de datos



Me situo en la base de datos, abro el menú en la esquina superior y selecciono la opción de importar.



Aquí selecciono el archivo bdgestion.sql para importarlo en la base de datos. En este punto, ya tengo las dos tablas “colección” y “usuarios”.

The screenshot shows the phpMyAdmin web interface. On the left, the database structure tree is visible, showing the 'bdgestion' database with tables 'coleccion' and 'usuarios'. The main panel displays the 'Filtros' (Filters) section with a search bar. Below this, a table lists the databases and tables in the system. The table has columns: Tabla, Acción, Filas, Tipo, and Coteja. The rows show 'coleccion' and 'usuarios' tables. Below the table, there is a summary row: '2 tablas', 'Número de filas', and '2 InnoDB utf8mtl'. At the bottom, there are links for 'Imprimir' (Print) and 'Diccionario de datos' (Data Dictionary).

Tabla	Acción	Filas	Tipo	Coteja
☐ coleccion	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8_s
☐ usuarios	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8_s
2 tablas Número de filas		2	InnoDB	utf8mtl

Creación del backend

En mi proyecto, creo la carpeta backend y me dirijo a ella.

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM> mkdir backend

Directorio: C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM

Mode                LastWriteTime         Length Name
----                -
d-----          20/11/2025   9:58             backend

PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM> cd backend
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend>
```

Dentro del backend uso el comando npm init para crear el fichero package.json, ya que lo necesita el node.js.

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults
.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (backend) backend
version: (1.0.0)
description: backend del proyecto
entry point: (index.js)
test command:
git repository:
keywords:
author: Hector
license: (ISC)
About to write to C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "backend del proyecto",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Hector",
  "license": "ISC"
}

Is this OK? (yes) yes

PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend>
```

Programar los endpoints del backend

Instalo las librerías necesarias

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend> npm install express
```

```
added 65 packages, and audited 66 packages in 2s
```

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend> npm install cors
```

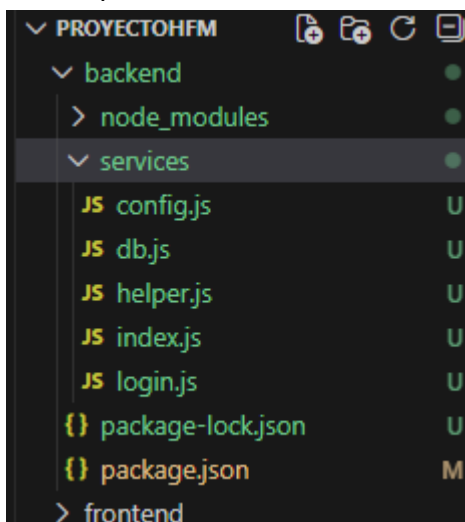
```
added 2 packages, and audited 68 packages in 964ms
```

Instalo la librería para poder trabajar con base de datos MySQL

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend> npm install mysql2
```

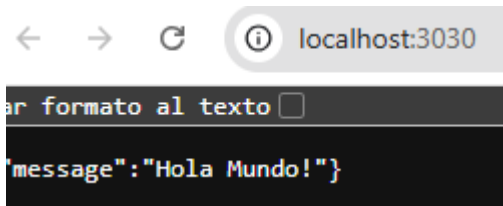
```
added 11 packages, and audited 79 packages in 1s
```

Muevo los archivos descargados en el Campus a la carpeta de backend (db.js y login.js en una carpeta nueva llamada services)



En el fichero package.json añado la línea para ejecutar index.js e inicio la API con npm start.

```
{ } package.json M X
backend > { } package.json > { } scripts > abc start
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "description": "backend del proyecto",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node index.js"
9    },
10   "author": "Hector"
```



Instalo el paquete nodemon para no tener que detener la API con cada cambio

```
PS C:\Users\alumnadoM\REACT PROJECTS\ProyectoHFM\backend> npm install nodemon
added 27 packages, and audited 106 packages in 7s
```

Y vuelvo a cambiar la línea de start en el package.json

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "nodemon index.js"
```

Llamar endpoints desde el frontend

Inserto el siguiente método en el frontend del Login y lo llamo en el handleSubmit.
(Reemplazandolo con el código anterior)

```
10 export default function Login() {
11   const dispatch = useDispatch();
12   const navigate = useNavigate();
13   const [user, setUser] = useState("");
14   const [passwd, setPasswd] = useState("");
15   const [alerta, setAlerta] = useState<{ tipo: "success" | "error" | ""; mensaje: string }>({
16     tipo: "",
17     mensaje: "",
18   });
19
20   async function isVerifiedUser() {
21     fetch(`http://localhost:3030/login?user=${user}&password=${passwd}`)
22       .then(response => response.json())
23       .then(response => {
24         console.log('Lo que nos llega de la base de datos: ');
25         console.log(response.data);
26         if (response.data.length !== 0) {
27
28           dispatch(
29             authActions.login({
30               name: response.data.nombre,
31               rol: response.data.rol
32             })
33           );
34           navigate("/home");
35         } else {
36           setAlerta({ tipo: "error", mensaje: "Usuario o contraseña incorrectos" });
37         }
38       })
39     }
40   }
41 }
```

Usando fetch accedo a llamadas HTTP. La respuesta se pasa al json, y si hay respuesta, comprueba con la base de datos. Si devuelve algo distinto de 0, significa que encontró el usuario y la contraseña y los guarda para mostrarlos en /home.

Comprobación del código

Con la API iniciada, introduzco los usuarios que están guardados en la base de datos para comprobar el funcionamiento.

← T →					id	nombre	login	password	rol
☐	 Editar	 Copiar	 Borrar	1	Patricia	patricia	123456789	admin	
☐	 Editar	 Copiar	 Borrar	2	usuario	user	123456789	user	

Página Login de Héctor Fleitas Martín
Sistema de acceso



Usuario *

patricia

Contraseña *

.....

ACCEDER

Bienvenido Patricia, tu rol es admin

CERRAR SESIÓN

Lo que nos llega de la base de datos: [Login.tsx:24](#)

```
{nombre: 'Patricia', rol: 'admin'} ⓘ Login.tsx:25
```

nombre: "Patricia"

rol: "admin"

▶ [[Prototype]]: Object

[Home.tsx:13](#)

```
{isAuthenticated: true, userName: 'Patricia', userRol: 'admin'} ⓘ
```

isAuthenticated: true

userName: "Patricia"

userRol: "admin"

▶ [[Prototype]]: Object

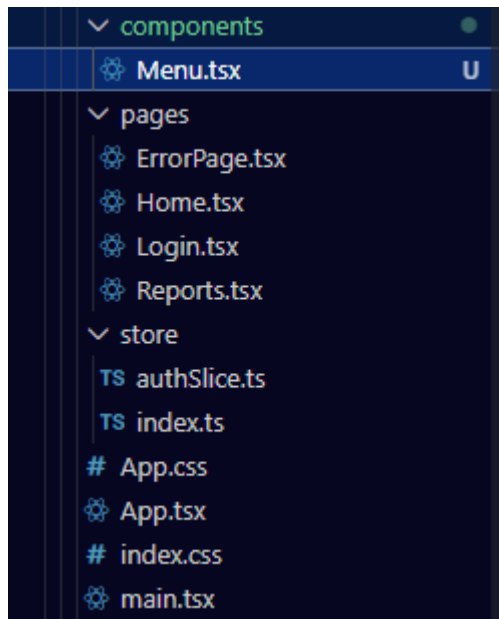
Y para "user"

Bienvenido usuario, tu rol es user

CERRAR SESIÓN

Creación de menú para todas las páginas

Creo el menu en Components:



Uso la siguiente estructura para home, y uso la misma para el resto

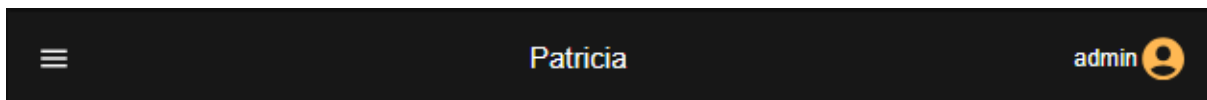
```
<List>

  <Link to="/home" style={{textDecoration: "none", color: "white" }}>
    <ListItem disablePadding>
      <ListItemButton>
        <ListItemIcon>
          <HomeIcon />
        </ListItemIcon>
        <ListItemText primary="Inicio" />
      </ListItemButton>
    </ListItem>
  </Link>
```

Las opciones Home, informes y salir están metidas en una lista.
En el return defino el TopBar e implemento el drawer

```
77     return (  
78         <>  
79         <AppBar position="static">  
80             <Toolbar>  
81                 /* Botón menú */  
82                 <IconButton color="inherit" edge="start" onClick={toggleDrawer(true)}>  
83                     <MenuIcon />  
84                 </IconButton>  
85  
86                 /* Usuario centrado */  
87                 <Typography sx={{ flexGrow: 1 }} variant="h6" align="center">  
88                     {userData.userName}  
89                 </Typography>  
90  
91                 /* Icono de usuario */  
92                 <Typography>{userData.userRol} </Typography>  
93                 {userData.userRol === "user" && (  
94                     <AccountCircleIcon fontSize="large" />  
95                 )}  
96                 {userData.userRol === "admin" && (  
97                     <AccountCircleIcon fontSize="large" color="secondary" />  
98                 )}  
99             </Toolbar>  
100         </AppBar>  
101  
102         <Drawer anchor="left" open={open} onClose={toggleDrawer(false)}>  
103             {DrawerList}  
104         </Drawer>  
105     </>  
106 );  
107
```

Si el usuario resulta ser "admin" se usa el color secundario del tema para el icono



Creación del formulario en Dashboard.tsx

Uso un formulario para insertar los datos, con textfields para rellenar y un botón para insertar.

```
return (  
  <Box sx={{ p: 3 }}>  
    <form  
      onSubmit={handleSubmit}  
      style={{  
        display: "flex",  
        flexDirection: "column",  
        gap: "15px",  
        maxWidth: "400px",  
        marginBottom: "40px"  
      }}  
    >  
      <TextField  
        label="Nombre"  
        name="nombre"  
        value={item.nombre}  
        onChange={handleChange}  
        required  
      />  
  
      <TextField  
        label="Marca"  
        name="marca"  
        value={item.marca}  
        onChange={handleChange}  
        required  
      />  
    </form>  
  </Box>  
)
```

```
<TextField
  label="Tipo"
  name="tipo"
  value={item.tipo}
  onChange={handleChange}
  required
/>

<TextField
  label="Precio"
  name="precio"
  type="number"
  value={item.precio}
  onChange={handleChange}
  required
/>

<Button variant="contained" type="submit">
  + INSERTAR DATOS
</Button>
</form>
```

Para indicar los tipos de los elementos (que precio sea number, por ejemplo) creo un interface antes, además de darles un estado inicial.

```
16 interface itemtype {
17   id?: number;
18   nombre: string;
19   marca: string;
20   tipo: string;
21   precio: number;
22 }
23
24 const itemInitialState: itemtype = {
25   nombre: "",
26   marca: "",
27   tipo: "",
28   precio: 0
29 };
```

Lista para los datos y su eliminación

Para mostrar los registros de la base de datos uso el componente Table de la librería @MUI
Contenedor de tabla (TableContainer)

La tabla en sí (Table)

La cabecera de la tabla (TableHead)

Componente fila (TableRow)
Las celdas (TableCell)
El cuerpo de la tabla (TableBody)

Uso la propiedad llamada aria-label para facilitar la lectura. El método handleDeleteItem() se verá más adelante.

```
<TableContainer component={Paper}>
  <Table aria-label="tabla coleccion">
    <TableHead>
      <TableRow>
        <TableCell>Borrar</TableCell>
        <TableCell>ID</TableCell>
        <TableCell>Nombre</TableCell>
        <TableCell>Marca</TableCell>
        <TableCell>Tipo</TableCell>
        <TableCell>Precio</TableCell>
      </TableRow>
    </TableHead>
    <TableBody>
      {tableData.map((row: itemtype) => (
        <TableRow key={row.id}>
          <TableCell>
            <Button
              color="error"
              onClick={() => handleDeleteItem(row)}
            >
              <DeleteForeverIcon />
            </Button>
          </TableCell>
          <TableCell>{row.id}</TableCell>
          <TableCell>{row.nombre}</TableCell>
          <TableCell>{row.marca}</TableCell>
          <TableCell>{row.tipo}</TableCell>
          <TableCell>{row.precio}</TableCell>
        </TableRow>
      ))}
    </TableBody>
  </Table>
</TableContainer>
```

Creación del fichero backend/services/items.js

Creo un fichero en el backend con el nombre items.js, que se encargará de crear las consultas que manejan la comunicación con la tabla coleccion de nuestra base de datos. Estas son las consultas a la base de datos por cada función:

```
5  async function insertData(req, res) {
6      const data = req.query;
7
8      const result = await db.query(`
9          INSERT INTO coleccion (nombre, marca, tipo, precio)
10         VALUES ('${data.nombre}', '${data.marca}', '${data.tipo}', '${data.precio}');
11     `);
12
13     );
14
15     return result.affectedRows;
16 }
17
18 async function getData(req, res) {
19     const rows = await db.query(`
20         SELECT id, nombre, marca, tipo, precio
21         FROM coleccion;
22     `);
23
24     const data = helper.emptyOrRows(rows);
25
26     return { data };
27 }
28
29 async function deleteData(req, res) {
30     const data = req.query;
31
32     const result = await db.query(`
33         DELETE FROM coleccion
34         WHERE id = ${data.id};
35     `);
36
37     return result.affectedRows;
38 }
39
40 module.exports = {
41     insertData,
42     getData,
43     deleteData
44 };
45
```

Y al final del fichero exporto las funciones getData, insertData y deleteData.

Añadir funcionalidad

En backend/index.js creo los endpoints para cada función de la página (insertar, mostrar y eliminar). Ejemplo del endpoint de insertar:

```
44 // INSERTAR
45 app.get('/addItem', async function (req, res, next) {
46   try {
47     res.json(await items.insertData(req, res));
48   } catch (err) {
49     console.error('Error while inserting item', err.message);
50     next(err);
51   }
52 });
```

En el frontend llamo al backend con la función fetch() cuando se pique el botón de Insertar. Si la respuesta del fetch es > 0, lanza un alert indicando que se ha insertado los datos correctamente.

```
<form
  onSubmit={handleSubmit}
```

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();

  fetch(
    `http://localhost:3030/addItem?nombre=${item.nombre}&marca=${item.marca}&tipo=${item.tipo}&precio=${item.precio}`
  )
    .then(res => res.json())
    .then(result => {
      if (result > 0) {
        alert("Datos guardados con éxito");
        setItem(itemInitialState);
        loadTableData();
      }
    });
};
```

Para el resto de funciones se hace exactamente lo mismo: Se añade endpoints y métodos para que funcionen.

Función para recargar la tabla con los datos:

```
const loadTableData = () => {
  fetch("http://localhost:3030/getItems")
    .then(res => res.json())
    .then(data => {
      setTableData(data.data);
    });
};
```

Esta función se lanza cuando se elimina o se inserta algún registro.

Función para eliminar registros:

```
const handleDeleteItem = (row: itemtype) => {  
  fetch(`http://localhost:3030/deleteItem?id=${row.id}`)  
    .then(res => res.json())  
    .then(result => {  
      if (result > 0) {  
        alert("Registro eliminado");  
        loadTableData();  
      }  
    });  
};
```

Se usa un icono que está dentro de la lista para borrarlo.

```
<TableCell>  
  <Button  
    color="error"  
    onClick={() => handleDeleteItem(row)}  
  >  
    <DeleteForeverIcon />  
  </Button>  
</TableCell>
```

Comprobación de la funcionalidad de la app

Entramos a nuestra app. La página que nos sale es la de Login.

Página Login de Héctor Fleitas Martín
Sistema de acceso



Usuario *

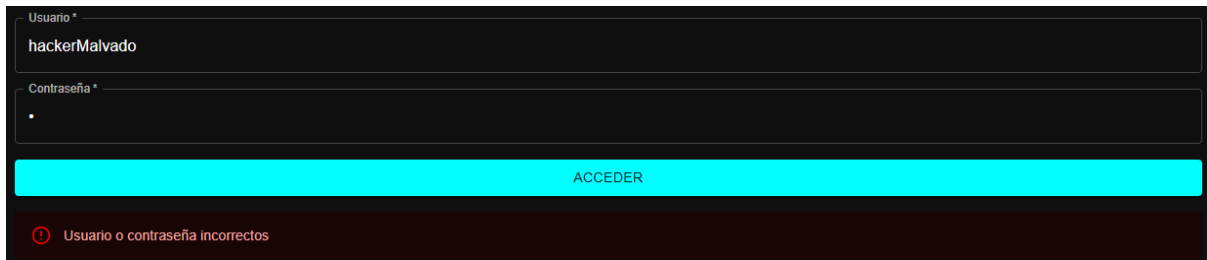
Contraseña *

ACCEDER

Aquí debemos introducir el usuario y contraseña que estén registrados en la base de datos.

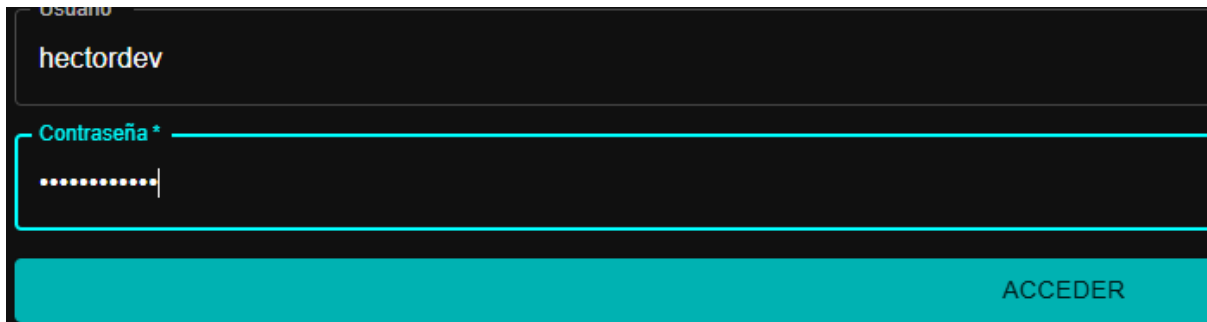
← T →				id	nombre	login	password	rol	
☐		Editar	 Copiar	 Borrar	1	Patricia	patricia	123456789	admin
☐		Editar	 Copiar	 Borrar	2	usuario	user	123456789	user
☐		Editar	 Copiar	 Borrar	3	Héctor	hectordev	merequetenge	admin

De lo contrario, nos dará una alerta.

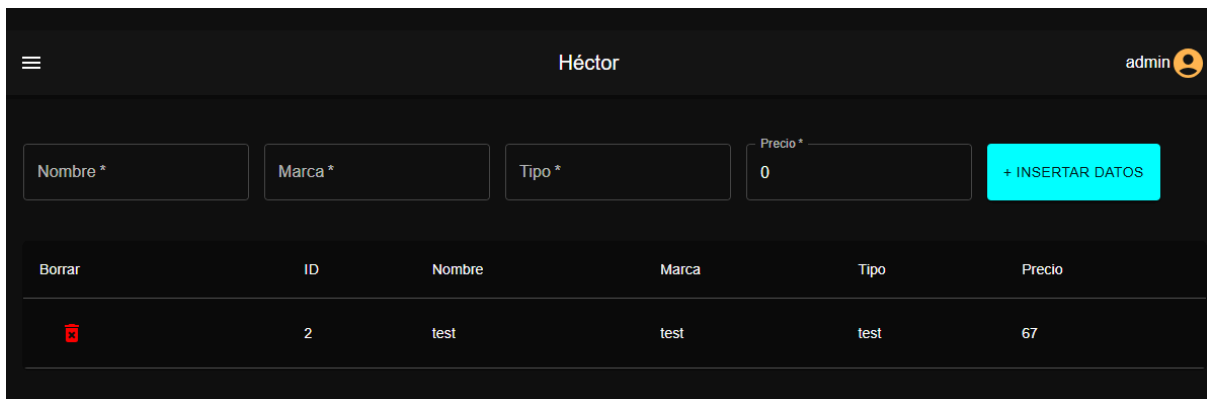


Formulario de inicio de sesión. El campo 'Usuario' contiene 'hackerMalvado' y el campo 'Contraseña' contiene un punto. El botón 'ACCEDER' es de color rojo. Debajo del botón, un mensaje de error indica: 'Usuario o contraseña incorrectos'.

Si escribimos alguno que sí esté registrado, accederemos a la página Home



Formulario de inicio de sesión. El campo 'Usuario' contiene 'hectordev' y el campo 'Contraseña' contiene puntos. El botón 'ACCEDER' es de color rojo.



Pantalla de inicio de sesión exitosa. El encabezado muestra el nombre 'Héctor' y el rol 'admin' con un ícono de usuario. Hay un menú de hamburguesa a la izquierda. El cuerpo de la página contiene un formulario para agregar datos con campos: 'Nombre *', 'Marca *', 'Tipo *' y 'Precio *' (valor 0). Un botón rojo '+ INSERTAR DATOS' está a la derecha. Debajo, una tabla muestra los datos ingresados:

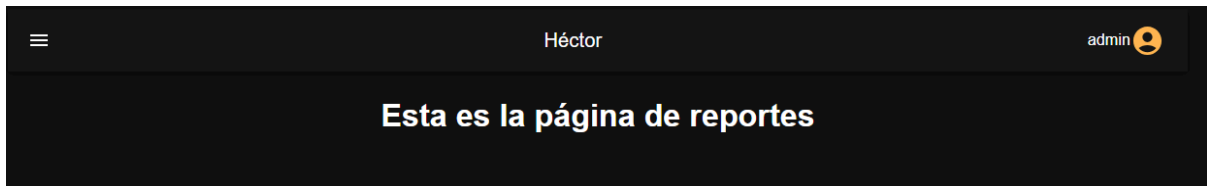
Borrar	ID	Nombre	Marca	Tipo	Precio
	2	test	test	test	67

Observamos que nuestro nombre aparece en el topbar, así como nuestro rol y el menú para acceder a otras páginas. Comprobamos que el drawer funciona. Si pulsamos en el ícono veremos este componente lateral



Menú lateral desplegado. Contiene tres opciones: 'Inicio' con un ícono de casa, 'Informes' con un ícono de gráfico de barras, y 'Salir' con un ícono de puerta de salida.

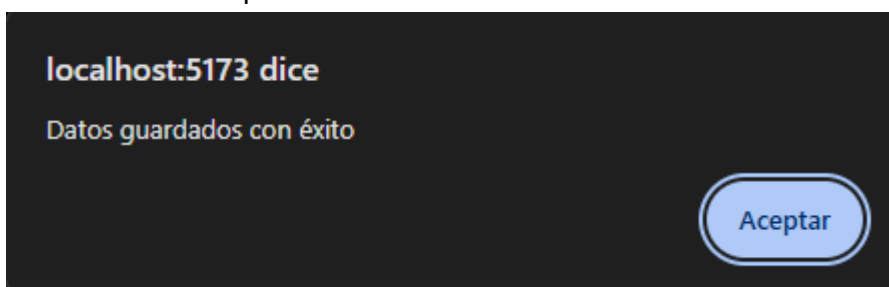
Desde aquí podremos ir a otras páginas como "Informes" o regresar a la página de Login.



Como se ve, el menú también estará presente aquí. Volvemos a Home y probamos a insertar datos. El campo de precio es numérico, así que no nos dejará escribir letras.

The form contains four input fields: 'Nombre *' with the value 'a', 'Marca *' with the value 'a', 'Tipo *' with the value 'a', and 'Precio *' with the value '0999'. The 'Precio *' field is highlighted with a red border, indicating its numeric nature.

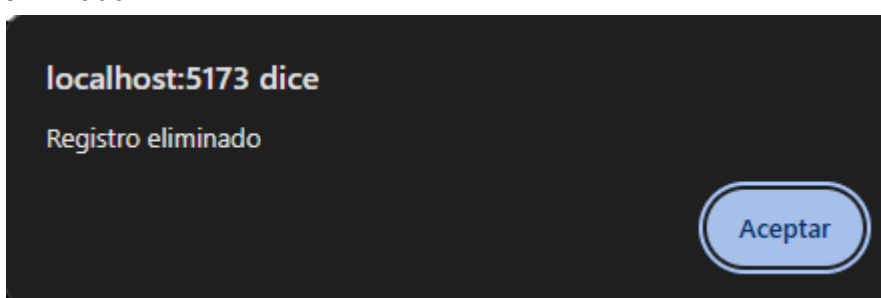
Tendremos una alerta para avisar de que se ha introducido correctamente el registro, y se actualizará la lista para ver los datos recién añadidos.



The form at the top has 'Nombre *', 'Marca *', 'Tipo *', and 'Precio *' (value '0') fields, followed by a red '+ INSERTAR DATOS' button. Below the form is a table with the following data:

Borrar	ID	Nombre	Marca	Tipo	Precio
	3	a	a	a	999

Al seleccionar el icono para borrar el registro, también saldrá una alerta de que se ha eliminado.



The table is empty, showing only the headers: 'Borrar', 'ID', 'Nombre', 'Marca', 'Tipo', and 'Precio'.

Por último, podemos comprobar desde la misma base de datos que se Sí se están insertando. Volvemos a insertar desde nuestra app

Nombre * Otro nuevo	Marca * No sé	Tipo * B	Precio * 069	+ INSERTAR DATOS
------------------------	------------------	-------------	-----------------	------------------

y revisamos.

← T →				id	nombre	marca	tipo	precio	
☐		Editar	 Copiar	 Borrar	4	test	test	test	999
☐		Editar	 Copiar	 Borrar	5	Nuevo	Ni idea	A	12.99
☐		Editar	 Copiar	 Borrar	6	Otro nuevo	No sé	B	69